```
app.Use(async (context, next) =>
{
    if (context.Request.Query.ContainsKey("stop"))
    {
        await context.Response.WriteAsync("Stop the world");
    }
    else
    {
        await next();
        logger.LogInformation("The world keeps turning");
    }
});
```

When registering middleware, the position is critical. If you were to place this middleware at the beginning of the pipeline, it would execute for every request and log the information message, unless the specified query string item is found. Suppose you were to register this middleware after the static files middleware. In that case, it would only execute and log requests for resources that are not static files, given that the static files middleware short-circuits the pipeline when returning a static file.

2.3.5 Middleware classes

All of the example middleware you have seen so far has been added as inline lambdas. This approach works for the simple middleware you have seen so far but falls short pretty quickly if your middleware involves any level of complexity, with both reusability and testability being adversely affected. At that point, you are likely to write your middleware in its own class.

There are two ways to implement middleware classes. The first option is to use a convention-based approach that has been part of ASP.NET Core since its beginning. The second option involves implementing the `IMiddleware` interface, which was introduced to ASP.NET Core 2.0 at the same time as Razor Pages.